

Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN

ANONYMOUS AUTHOR(S)

AI-RAN servers run the uplink physical layer and AI inference on the same GPUs. Neural receivers decode transport blocks (TBs) that the conventional receiver fails, but their value and their cost on a shared GPU are not established. We compare public neural receivers with the cuPHY receiver at the same LDPC iteration count and find that the real-time model helps only when the channels of paired users are correlated, while a larger model decodes 15-67% of the failed TBs on every channel we test. The larger model takes 5.3 ms per slot, twice the uplink period. This paper presents Backstop, which runs the larger model only for failed TBs under a later deadline, runs AI inference at a low GPU priority, and grants it in short pieces that keep the layer-1 deadline, the recovery deadline, and a free neural receiver for the next failed TB. On four A100 GPUs with 16 cells, Backstop recovers 63-66% of the failed TBs on the low-correlation channel and 38% with mixed channels. Under a load that changes every two seconds, Backstop serves 46.2k tokens/s while keeping 98.8% of the recoveries: $8.0\times$ a fixed GPU share, $2.0\times$ a share that follows the load, and $1.3\times$ a share that follows the load with low-priority AI, each at the same or a lower recovery level.

CCS Concepts: • **Computer systems organization** → **Real-time systems**; • **Networks** → *Network resources allocation*.

Additional Key Words and Phrases: AI-RAN, GPU scheduling, neural receiver, MPS, admission control

ACM Reference Format:

Anonymous Author(s). 2027. Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN. In *Proceedings of ACM SIGMETRICS 2027 (SIGMETRICS '27)*. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Radio access networks (RANs) increasingly run baseband processing on GPUs, and AI-RAN platforms run radio physical-layer (PHY) processing and AI inference on the same GPUs [3, 13, 18, 21]. To use the GPU time that radio processing leaves idle, these platforms add AI services, such as large language model (LLM) inference, next to deadline-critical PHY tasks [12, 20]. Neural receivers are a PHY workload that these platforms now run on the GPU [13, 24]. A neural receiver replaces channel estimation, equalization, and demapping with a learned model and decodes transport blocks (TBs) that a conventional receiver fails [6, 10, 24]. Each TB that it decodes avoids a retransmission and frees an uplink slot.

The challenge stems from two facts that we measure in this paper. First, the neural receiver that helps is expensive. With the same number of LDPC iterations in both receivers, the real-time model of the public NVlabs receivers [7, 24] decodes more TBs than the cuPHY receiver only when the channels of the paired users are correlated. The larger model decodes more TBs on every channel we test, and it takes 5.3 ms per slot against an uplink period of 2.5 ms. Thus, it cannot run for every slot. Second, the larger model shares the GPU with the conventional receivers of all cells and with AI inference, and the existing GPU controls do not protect it. NVIDIA Multi-Process Service (MPS) caps the GPU threads of a process, but the cap is fixed when the process starts [14]. A cap that protects the radio work at full load leaves the GPU idle when the load drops. A low MPS priority for AI keeps the layer-1 deadline, but it loses 8% of the recoveries at full load. We find that these recoveries are lost because a failed TB cannot wait: when AI slows the neural receivers, the next failed TB finds none of them free.

Existing systems cover one side of this problem. As Table 1 summarizes, vRAN and AI-RAN resource managers [3, 8, 12, 18–21] share GPUs between radio processing and other workloads by

Table 1. What the closest public designs run on the shared GPU.

System	Radio work on the GPU	How AI shares the GPU
YinYangRAN, CloudRIC, Concordia	LDPC decoding or queued PHY jobs	GPU share or reserved capacity, changed at intervals of one second or more
DARIS, REEF, XSched, Orion, SMEC, nvtaskset	None; generic GPU tasks	Kernel, context, MPS, or partition priority without radio deadlines
ARCHES, OCUDU, neural receivers	Neural channel estimator or neural receiver	No AI workload on the same GPU
Backstop	Conventional receiver for every TB and a neural receiver for failed TBs	Low-priority AI pieces granted against both deadlines and the free neural receivers

a GPU share or by reserved capacity, and their radio work is LDPC decoding or the conventional PHY. GPU scheduling systems [4, 5, 9, 22, 23, 25] prioritize or preempt GPU kernels and contexts without radio deadlines. Neural receiver studies and PHY platforms [6, 10, 15, 17, 24] design the receiver or switch between PHY models, and they do not share the GPU with an AI workload. However, none of them runs a neural receiver next to AI inference on one GPU while keeping the deadlines of both radio paths.

In this paper, we present Backstop, a GPU sharing scheme for AI-RAN servers that uses a neural receiver as a second receiver for the TBs the conventional receiver fails. Specifically, Backstop introduces 1) a recovery path that sends a failed TB to the neural receiver only when the number of failed code blocks predicts a successful decode, 2) two deadlines per TB, the layer-1 deadline for the conventional receiver and a later recovery deadline before the retransmission is scheduled, and 3) AI admission in bounded pieces at low priority, where the controller allows a piece only if every running and waiting radio job still meets its deadline and enough neural receivers stay free for the next failed TB. Our evaluation on four A100 GPUs with 16-32 cells, the cuPHY receiver, the public NVlabs neural receivers, and Qwen2.5 prefill requests shows that Backstop recovers 63-66% of the TBs the conventional receiver fails on the low-correlation channel and 38% with mixed channels. With mixed channels and a load that alternates between full and half every two seconds, Backstop serves 46.2k tokens/s within the 200 ms limit and keeps 98.8% of the recoveries. A fixed 10% GPU share keeps the same recoveries at 5.8k tokens/s. A share that follows the load with no switching cost reaches 23.6k tokens/s, and 34.5k with low-priority AI, both with fewer recoveries. The layer-1 misses of Backstop stay at the level of the run without AI. The free-receiver rule needs several neural receivers: with one GPU, Backstop equals a 70% share with low-priority AI, and we report this setting as well.

2 Background and Motivation

2.1 AI-RAN on shared GPUs

An AI-RAN server decodes the uplink of many cells on GPUs and serves AI requests with the GPU time that is left [3, 13].

Uplink Timeline. With the TDD pattern DDDSU at 30 kHz subcarrier spacing, every cell has one uplink slot per 2.5 ms. The received samples of a slot are ready 0.5 ms after the slot starts, and the layer-1 result is due 4.5 ms after the slot starts, which leaves 4.0 ms for decoding. A TB that fails is retransmitted three uplink slots later, 7.5 ms after the original slot: the grant goes out in the special slot before that uplink slot with a scheduling offset of one slot, which the PUSCH preparation time of 12 symbols allows [2].

GPU Sharing. MPS runs the processes of one GPU concurrently and caps the active threads of each process [14]. YinYangRAN changes the share between LDPC decoding and other workloads

Table 2. TBs decoded at 20 LDPC iterations in both receivers. The percentage is the share of the TBs cuPHY fails that the neural receiver decodes; the time is the GPU time of the model alone.

Channel of the two users	TBs	cuPHY	Real-time model (1.35 ms)	Larger model (4.29 ms)
Low correlation, 2-3 dB	600	515	484 (2%)	572 (67%)
Urban micro, 1-6 dB	600	425	420 (2%)	451 (15%)
Medium correlation, 8-18 dB	600	321	372 (21%)	not measured
High correlation, 10-16 dB	1,400	946	1,099 (38%)	1,159 (51%)

with a learned predictor at intervals of at least one second [18], and CloudRIC pools GPUs for LDPC decoding across distributed units [19]. The share of an MPS client cannot change after the client starts. Therefore, a system that follows the load must restart the client or keep one loaded client per share.

2.2 When does a neural receiver help?

We compare the public NVlabs neural receivers [6, 7, 24] with the cuPHY PUSCH receiver [13] on the same received slots. Two users transmit on the same 273 PRBs with 16QAM, and the base station has four antennas. The slots come from the NVlabs link simulation [11]; cuPHY decodes them with MMSE equalization and time interpolation.

Same LDPC Effort. A receiver comparison is valid only when both receivers run the same number of LDPC iterations. cuPHY uses 10 iterations for the TB sizes in our setup, and the evaluation code of the neural receivers uses 20. With 10 against 20 iterations, the real-time model appears to decode 25-35% of the TBs that cuPHY fails on the low-correlation channel. With 20 iterations on both sides, the figure is 2%. Raising cuPHY from 10 to 20 iterations alone recovers 152 of 158 failed single-user TBs, at 15-19% more receiver time for TBs of 13-20 code blocks. Therefore, every result in this paper uses 20 iterations in both receivers. Table 2 shows the comparison.

The Model That Helps Is Too Slow for Every Slot. The real-time model decodes more TBs than cuPHY only when the channels of the two users are correlated. The larger model decodes more TBs on all three channels and loses none of the TBs that cuPHY decodes on two of them. It takes 5.3 ms per slot alone and 6.3 ms next to the cell workload, against an uplink period of 2.5 ms; cuPHY takes 0.8-0.9 ms. When we send every slot of four cells to the larger model, 61% of the slots find no free GPU in time and 89.4% of the TBs are decoded. When we send only the TBs that cuPHY fails, the neural receiver runs for 25% of the slots, the decode result is the same offline, and 94.5-95.1% of the TBs are decoded in the timed system.

Failed Code Blocks Predict the Outcome. The conventional receiver reports how many code blocks of a TB failed. On the urban micro channel, 87% of the failed TBs have every code block failed and the neural receiver decodes 2% of them. It decodes 93% of the TBs in slots with at most nine failed code blocks. On the low-correlation channel, it decodes 97% of the failed TBs in slots with one to three failed code blocks and 30% in slots with ten or more.

This motivates Backstop's design: run the larger model only for failed TBs that it is likely to decode, give that run a deadline later than the layer-1 deadline, and admit AI only where both deadlines hold.

3 Backstop Design

3.1 Overall architecture

Backstop runs four kinds of processes on each server (Figure 1). A conventional receiver process per cell decodes the TBs of its cell with cuPHY every 2.5 ms. A neural receiver process per GPU

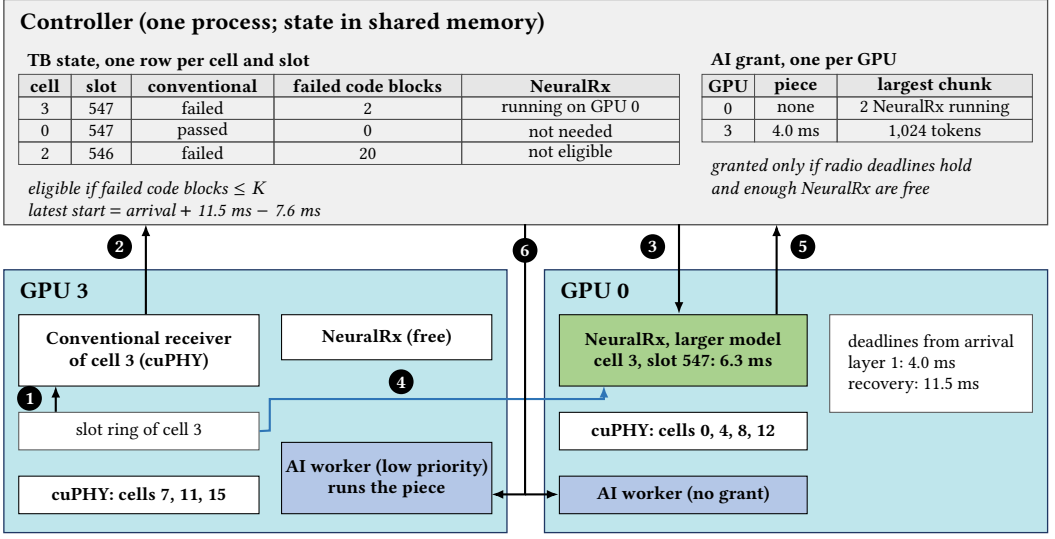


Fig. 1. Overall architecture of Backstop with the values of one recovery in a measured run (two of the four GPUs are shown). Numbers mark the steps of the text.

decodes one TB at a time. An AI worker per GPU runs AI inference in pieces. One controller assigns failed TBs to neural receivers, grants AI pieces, and places AI requests on GPUs. All processes read one shared state in memory and one clock.

The steps for one slot are as follows. ❶ The samples of a slot arrive in the slot ring of the cell, which stays in the GPU memory of its conventional receiver, and cuPHY decodes them. ❷ The receiver writes the CRC result and the number of failed code blocks to the TB state. ❸ The controller assigns an eligible failed TB to a free neural receiver before the latest start time of the TB. ❹ Every neural receiver process maps the slot rings of all cells. Thus, the neural receiver copies the slot from the GPU of the cell and decodes it on its own GPU. ❺ A result that arrives before the recovery deadline cancels the retransmission. ❻ In every round the controller writes one AI grant per GPU, and the AI worker runs only inside the granted piece.

3.2 Recovery path

Two Deadlines. Every TB has a layer-1 deadline of 4.0 ms after its samples arrive, which the conventional receiver must meet. A TB that fails has a *recovery deadline*: if the neural receiver decodes it before this deadline, the retransmission is cancelled. A recovery deadline of 6.5 ms keeps the retransmission slot of the conventional path. The larger model does not fit into 6.5 ms, and Backstop uses 11.5 ms, which delays the retransmission of a TB that is not recovered by two uplink slots (5 ms).

Which TBs Are Recovered. A failed TB becomes a candidate only if its slot has at most K failed code blocks. The controller computes the latest start time of a candidate as its arrival plus the recovery deadline minus the time bound of the neural receiver. It assigns candidates in order of latest start time to a free neural receiver, preferring a GPU with no other neural receiver running and no AI piece running. A candidate that passes its latest start time is dropped and its TB is retransmitted. A neural receiver result counts only if the CRC passes, the payload is correct, and the run ends before the recovery deadline.

3.3 AI admission in bounded pieces

AI Units. The AI worker splits the prefill of an LLM request into units of one layer over a chunk of 128, 512, or 1,024 tokens, each compiled as one GPU graph with a measured time bound. A larger chunk is faster per token and holds the GPU longer once it starts.

Low Priority. The AI worker runs at a lower MPS priority than the radio processes [14] and has no cap on its share. Next to low-priority AI units of 1,024 tokens, the conventional receivers of four cells finish in 3.4 ms at the 99th percentile, against 2.9 ms alone, and the neural receiver takes 7.9 ms at the median, against 6.3 ms alone. Low priority keeps the layer-1 deadline. It does not keep the recoveries, as Section 4.2 shows.

Why Recoveries Are Lost. A candidate cannot wait for a neural receiver: it must start before its latest start time, 3.9 ms after its samples arrive. If every neural receiver is running at that time, the candidate is dropped. Without AI, 4% of the candidates are dropped. AI next to a neural receiver makes its run longer, and the neural receiver is free later. In our runs, more than 90% of the recoveries that a policy with low-priority AI or a share of at most 30% loses are candidates that found no free neural receiver, and less than 10% are neural receiver runs that finish after the recovery deadline. Therefore, a rule that only checks the deadline of the running neural receiver is not enough. A longer run costs a recovery only when the other neural receivers are busy as well.

Grants. The AI worker submits units only inside a piece that the controller grants. For each GPU the controller picks the largest chunk size c that meets all of the following and grants a piece of at most 4 ms of AI work:

- *Free neural receivers.* If a neural receiver runs on this GPU, at least R neural receivers of the server are free. With four neural receivers we use $R = 3$: AI runs next to a neural receiver only while no other neural receiver is running.
- *Running neural receiver.* For every neural receiver running on this GPU, its start time plus $B(c)$ is before its recovery deadline, where $B(c)$ is the time bound of the neural receiver next to AI units of size c .
- *Conventional receivers.* If size c is not allowed next to the conventional receivers, no cell of this GPU is decoding and the piece ends before the next slot arrives.
- *Neural receiver about to start.* If a TB of this GPU may still become a candidate, the piece ends before the latest start time of that TB under $B(c)$.
- *Waiting candidates.* No candidate is waiting for a free neural receiver.

If no size meets the conditions, the GPU gets no AI in this round. The worker keeps the chunk size of a request stable by choosing the largest size that was allowed for most of the last 20 ms. A piece that is running when a condition stops holding runs to its end; at low priority a piece takes 2.3 ms at the median and 8.7 ms at the 99th percentile next to radio work.

Figure 2 shows the rule on a measured run. At the start, one neural receiver runs on GPU 3 and AI runs next to it. At 9 ms two neural receivers start on GPU 0 and GPU 1. Two neural receivers are free, the AI worker of GPU 0 stops, and both runs finish in 6.2-6.3 ms. The AI workers of GPU 2 and GPU 3 continue. At 19 ms one neural receiver runs alone on GPU 1, and AI runs next to it from 23 ms.

AI Requests. The controller predicts when a new request would finish from the tokens that wait on each GPU and the recent prefill rate of that GPU. It places the request on the GPU that finishes it first and rejects it if no GPU finishes it within the limit. Every baseline uses the same placement and rejection.

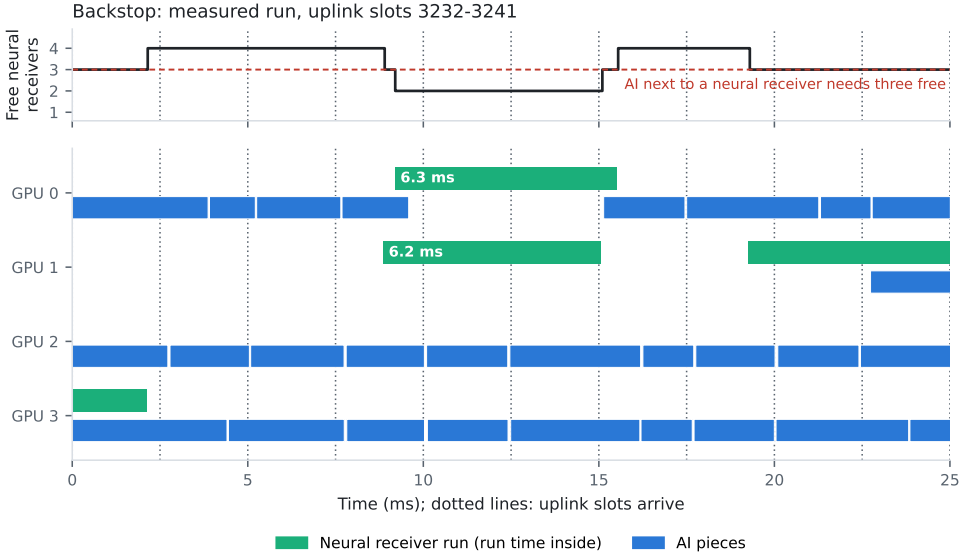


Fig. 2. Measured timeline of the four GPUs under Backstop (16 cells, full load). The upper panel counts the free neural receivers.

3.4 Time bounds and implementation

The bounds come from measurements on the evaluation hardware with all cells running and low-priority AI of one unit size running continuously. With 16 cells, the larger neural receiver takes 6.3 ms at the median, and the bound is 7.6 ms alone and 8.1-8.7 ms next to AI units, the 99th percentile plus a margin. With four cells per GPU, the conventional receivers finish in 3.7-4.05 ms at the 99.9th percentile next to continuously running units of the three sizes, and we allow all three next to them. With five cells per GPU they finish in 3.7 ms next to 128-token units and in 4.3-4.5 ms next to larger units, and we allow only 128-token units. The neural receiver path is the public TensorRT export of the NVlabs model followed by rate recovery, LDPC decoding, and CRC from cuPHY. We replace the least-squares channel estimation call with one multiplication of the pilot samples and call the cuPHY bit-level functions on buffers allocated once, which shortens the path of the real-time model from 3.08 to 2.36 ms with the same decoded TBs. The run-time path of the real-time model agrees with the NVlabs evaluation on 794 of 800 TBs. Backstop is 3.0k lines of Python over cuPHY, TensorRT, and PyTorch.

4 Evaluation

4.1 Evaluation setup

Hardware. One node with four NVIDIA A100 80 GB GPUs under MPS and 64 CPU cores.

Scope. All measurements are end-to-end runs on the GPUs: cuPHY decodes every TB, the neural receiver decodes real received slots, and the AI worker runs real prefill. The slots are generated in advance; there is no radio unit and no layer-2 stack.

Workloads. Sixteen cells run on four GPUs, four per GPU, unless stated. One cell per GPU carries two users on the same 273 PRBs (16QAM, 20 code blocks per slot); the other cells carry one user with two layers. The two-user slots come from three channels of the NVlabs link simulation: low

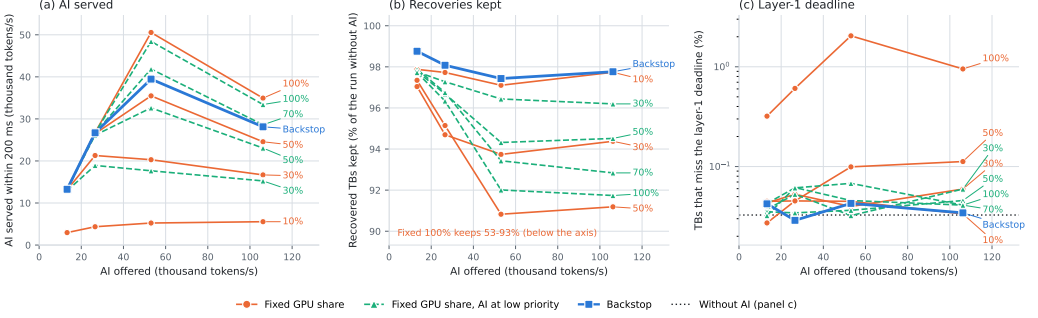


Fig. 3. AI served, recoveries kept, and layer-1 misses against the AI offered (16 cells at full load, three seeds). Labels give the MPS shares.

correlation at 2-3 dB, high correlation at 13-16 dB, and urban micro [1] at 4-6 dB. Both receivers run 20 LDPC iterations, the neural receiver is the larger NVlabs model, $K = 9$, and the recovery deadline is 11.5 ms. AI requests are Qwen2.5-1.5B [16] prefill requests of up to 1,024 tokens with a 200 ms limit, offered at 32 requests/s per GPU (53k tokens/s for the server) unless stated. A request that finishes after the limit does not count. Runs last 10-40 s; values are means over two or three seeds.

Baselines. Every policy uses the same recovery path, the same AI requests, and the same placement and admission of AI requests. The policies differ in when AI may use the GPU.

- *Fixed share* gives the AI worker a fixed MPS share, a static split of the GPU between radio and AI. A share of 100% is plain MPS with no protection.
- *Fixed share with low-priority AI* adds a low MPS priority for the AI worker, the mechanism that SMEC applies on an edge server [25]. A share of 100% is low priority alone.
- *Share that follows the load* keeps one AI worker per share loaded on every GPU and makes the worker of a lower share active when the radio load of the GPU is higher, as YinYangRAN changes the share [18]. It has no switching cost, and the waiting requests pass to the active worker. We run it with the load known at once and known one second late, the minimum reconfiguration interval of YinYangRAN, and with and without low-priority AI.
- *Radio-idle only* runs AI only while the GPU has no radio work and ends each piece before the next slot arrives, as systems that run best-effort GPU work around a high-priority job [9, 23].

The baselines are the mechanisms of these systems on our code, not the systems. We report the recoveries of a run as a share of the recoveries of the run without AI on the same seed.

4.2 AI load at a steady full load

Figure 3 shows the AI served, the recoveries kept, and the layer-1 misses for 13k to 106k tokens/s offered with all 16 cells at full load. Table 3 gives the values at 53k.

Layer-1 Deadline. Backstop misses the layer-1 deadline for 0.03-0.04% of the TBs at every AI load, the level of the run without AI (0.03%). The fixed shares miss more as the share grows: 0.10-0.11% at 50% and 0.3-2.0% at 100%, where the conventional receivers finish in 5.2 ms at the 99.9th percentile. The policies with low-priority AI stay at 0.03-0.07%.

AI Served. At 13k and 27k tokens/s offered, Backstop serves every request. At 53k, it serves 39.5k tokens/s and keeps 97.4% of the recoveries. The policies that keep as many recoveries are the fixed 10% share, which serves 5.2k tokens/s, and the 30% share with low-priority AI, which serves 17.7k.

Table 3. Steady full load, 16 cells, 53k tokens/s offered, three seeds.

Policy	AI tokens/s within 200 ms	Recoveries kept	Layer-1 misses
Backstop	39.5k (37.4-41.9k)	97.4% (97.1-97.6%)	0.04%
Fixed 10%	5.2k (5.1-5.5k)	97.1% (96.8-97.7%)	0.05%
Fixed 30%	20.3k (19.7-20.9k)	93.7% (93.2-94.8%)	0.04%
Fixed 50%	35.5k (34.2-37.5k)	90.8% (89.1-92.8%)	0.10%
Fixed 100% (no protection)	50.6k	53.2%	2.04%
Fixed 30%, low-priority AI	17.7k (17.6-17.8k)	96.4% (96.2-96.9%)	0.03%
Fixed 50%, low-priority AI	32.6k (31.3-34.4k)	94.3% (94.0-94.6%)	0.04%
Fixed 70%, low-priority AI	41.9k (39.7-44.3k)	93.4% (92.5-94.4%)	0.05%
Low-priority AI, no cap	48.4k (46.3-50.8k)	92.0% (90.7-93.3%)	0.07%
Radio-idle only	4.7k (4.2-4.9k)	99.7% (99.5-99.9%)	0.03%
No AI	0	100%	0.03%

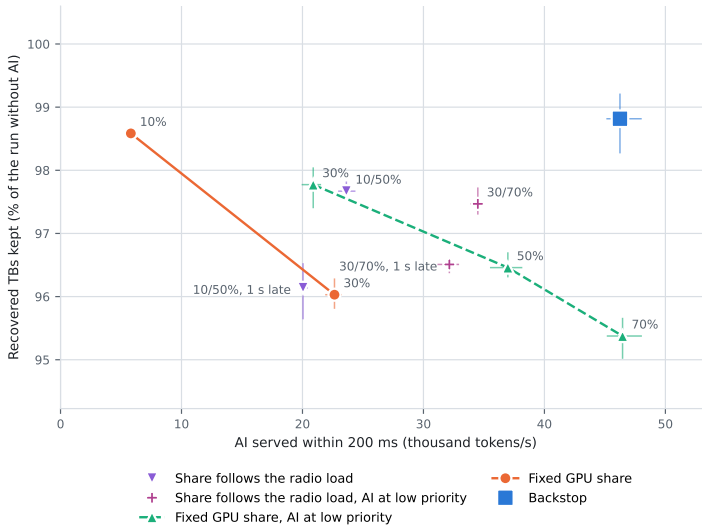


Fig. 4. AI served and recoveries kept when the radio load alternates between full and half every 2 s (16 cells, three seeds; bars show the range over seeds). Labels give the MPS shares.

Backstop serves 7.6× and 2.2× more. The 70% share with low-priority AI serves 41.9k tokens/s and loses 6.6% of the recoveries, 2.5× more than Backstop. Low priority alone serves 48.4k and loses 8.0%. Running AI only while the GPU has no radio work keeps 99.7% and serves 4.7k tokens/s, because four cells and the neural receiver leave a GPU idle for a small part of the time.

Overload. At 106k tokens/s offered, every policy serves less than at 53k: the admission accepts shorter requests, which take more GPU time per token. Backstop serves 28.1k tokens/s at 97.8%, and the 70% share with low-priority AI serves 28.6k at 92.8%.

4.3 AI served under a changing load

Alternating Load. The radio load alternates every two seconds between full load and each cell active with probability 0.5. Figure 4 and Table 4 show AI served against recoveries kept. Backstop serves 46.2k tokens/s and keeps 98.8% of the recoveries. The fixed 10% share keeps 98.6% and serves

Table 4. Changing load, 16 cells, load alternating between full and half every 2 s, three seeds (range over seeds in parentheses).

Policy	AI tokens/s within 200 ms	Recoveries kept
Backstop	46.2k (45.2-48.0k)	98.8% (98.3-99.2%)
Fixed 10%	5.8k (5.7-5.9k)	98.6% (98.5-98.6%)
Fixed 30%	22.6k (21.9-23.2k)	96.0% (95.8-96.3%)
Share follows load, 10/50%	23.6k (23.0-24.3k)	97.7% (97.6-97.8%)
Share follows load, 10/50%, 1 s late	20.0k (19.8-20.4k)	96.1% (95.7-96.5%)
Fixed 30%, low-priority AI	20.9k (20.0-21.5k)	97.8% (97.4-98.0%)
Fixed 70%, low-priority AI	46.5k (45.2-48.0k)	95.4% (95.0-95.7%)
Share follows load, 30/70%, low-priority AI	34.5k (33.9-34.9k)	97.5% (97.3-97.7%)
Share follows load, 30/70%, low-priority AI, 1 s late	32.1k (31.2-32.9k)	96.5% (96.4-96.6%)

5.8k tokens/s, 8.0× less. The share that follows the load serves 23.6k tokens/s at 97.7%, 2.0× less, and 20.0k at 96.1% when it sees the load one second late. With low-priority AI, the share that follows the load serves 34.5k tokens/s at 97.5%, 1.3× less than Backstop, and 32.1k at 96.5% one second late. The fixed 70% share with low-priority AI serves as much AI as Backstop and loses 4.6% of the recoveries, 3.8× more. The layer-1 misses are 0.02-0.04% for every policy in the table, against 0.01% without AI. With a change every 0.2 s and every 5 s (two seeds), Backstop serves 48.4k and 46.2k tokens/s and keeps 98.7% and 98.6%. The share that follows the load judges the load over 100 ms; with a change every 0.2 s and no low priority, it keeps 96.8% of the recoveries, and one second late it misses the layer-1 deadline for 0.13% of the TBs.

Random Load Steps. In Figure 5 the share of active cells jumps between 25, 50, 75, and 100% at random times 0.5-1.5 s apart, and the run lasts 40 s. Backstop serves 48.6k tokens/s and keeps 99.3% of the recoveries (two seeds). The share that follows the load with three shares (30, 50, and 70%) and low-priority AI serves 40.4k tokens/s at 98.5%, and 39.5k at 97.2% one second late. The fixed 70% share with low-priority AI serves 48.1k tokens/s and loses 2.8% of the recoveries, 4× more than Backstop. The difference comes from the periods of full load: Backstop serves 41.6k tokens/s there and keeps 99.0%, the fixed 70% share serves 43.3k and keeps 93.9%, and the share that follows the load drops to its 30% share and serves 20.2k at 96.9%. The fixed 10% share keeps 99.3% and serves 5.9k tokens/s, 8.2× less than Backstop.

Bursts. When every cell switches on and off by itself with a mean busy run of 20 ms or 2 s (16 cells, half load on average, two seeds), Backstop serves 52.4k and 51.6k tokens/s and keeps 99.6% and 99.3% of the recoveries. The 70% share with low-priority AI serves 51.1k and 50.4k tokens/s and keeps 99.3% and 98.2%. At this load the neural receivers are needed half as often as at full load, and low priority alone loses few recoveries. With 2 s runs, the layer-1 misses are 0.17% without AI and 0.17-0.26% for every policy, and we do not compare the policies on the layer-1 deadline in this setting. When AI requests arrive in bursts at a steady full radio load (inter-arrival times with a coefficient of variation of 4), Backstop serves 26.8k tokens/s at 98.3%, the fixed 10% share 3.7k at 98.0%, and the 70% share with low-priority AI 27.6k at 95.2%.

4.4 Neural receiver demand

Table 5 raises the neural receiver demand by making more cells two-user cells. Under Backstop, the recovered TBs grow with the demand, from 124 to 355 TBs/s, and stay at 96.9-99.0% of the run without AI; 99.9-100% of the neural receiver runs end before the recovery deadline. Backstop gives the GPU time back as the demand grows: it serves every AI request when no cell needs the neural receiver and 14.6k tokens/s with eight two-user cells. A fixed share cannot give the time back. With

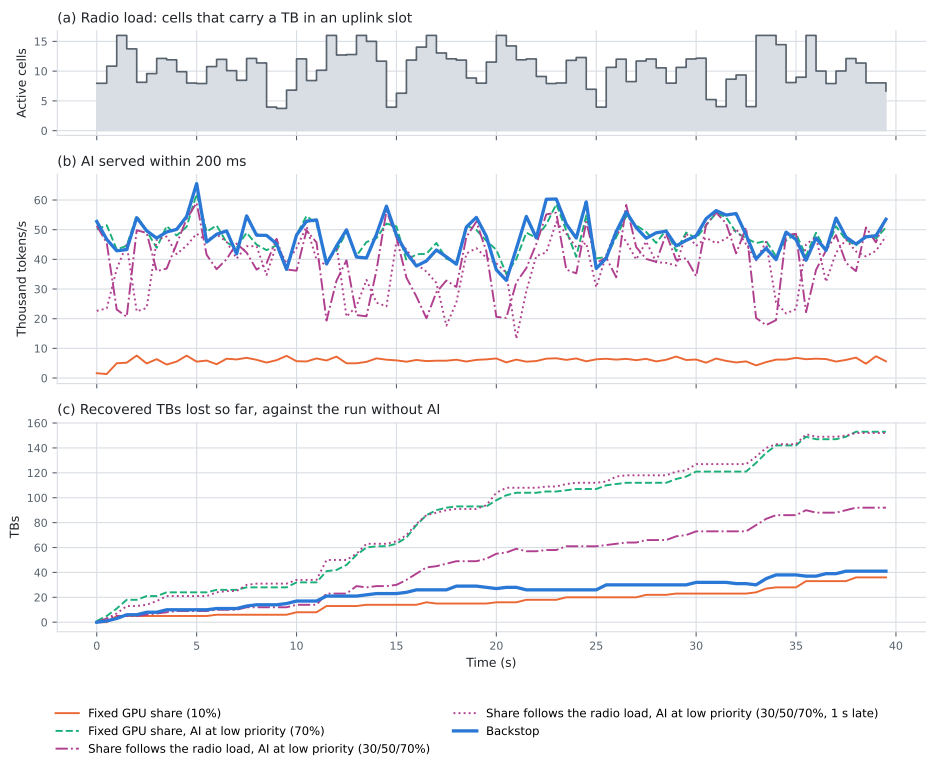


Fig. 5. Radio load that jumps between 25, 50, 75, and 100% at random times (16 cells, one of two seeds): AI served per 0.5 s and the recoveries lost so far.

Table 5. Two-user cells among 16 cells at full load, 53k tokens/s offered (two seeds; four cells: three). Each cell shows AI tokens/s and recoveries kept.

Two-user cells	Recovered TBs/s, no AI	Backstop	Fixed 10%	30%, low priority	70%, low priority
0	0	53.9k	6.0k	28.4k	53.4k
2	125	51.1k, 99.0%	5.5k, 98.8%	21.4k, 98.1%	48.5k, 97.8%
4	202	39.5k, 97.4%	5.2k, 97.1%	17.7k, 96.4%	41.9k, 93.4%
6	282	25.9k, 97.8%	4.8k, 98.3%	14.0k, 96.8%	32.1k, 94.8%
8	366	14.6k, 96.9%	4.4k, 97.6%	11.3k, 95.4%	25.4k, 89.3%

eight two-user cells, the 70% share with low-priority AI loses 10.7% of the recoveries, and the fixed 10% share serves 3.3× less AI than Backstop. The layer-1 misses of Backstop are 0.03-0.05% in all five settings.

4.5 More cells and more GPUs

Cells. Table 6 keeps the offered radio load and spreads it over 32 and 48 cells, where each cell carries a TB in a slot with probability 0.5 and 0.33. Backstop serves 37.7-39.5k tokens/s and keeps 97.4-98.6% of the recoveries with 16, 32, and 48 cells, 7.4-7.6× the fixed 10% share and 2.2× the 30% share with low-priority AI. With 48 cells, up to 12 cells of a GPU are active in one slot, and the layer-1 misses are 0.09% without AI and 0.09-0.14% for every policy; Backstop does not reduce

Table 6. Scale, 53k tokens/s offered per four GPUs (two seeds; 16 cells: three). Each cell shows AI tokens/s and recoveries kept.

Setting	Backstop	Fixed 10%	30%, low priority	70%, low priority
16 cells, full load	39.5k, 97.4%	5.2k, 97.1%	17.7k, 96.4%	41.9k, 93.4%
32 cells, half load	38.0k, 98.6%	5.1k, 99.2%	16.9k, 99.0%	40.0k, 96.3%
48 cells, one third load	37.7k, 98.3%	5.1k, 99.3%	16.8k, 98.6%	39.4k, 96.5%
20 cells, full load	27.2k, 96.1%	5.1k, 99.3%	17.0k, 96.1%	40.5k, 94.2%
1 GPU, 4 cells	8.8k, 100%	1.1k, 100%	3.7k, 100%	8.4k, 100%
2 GPUs, 8 cells	19.4k, 98.5%	2.3k, 98.7%	8.1k, 99.0%	19.4k, 96.9%

Table 7. Rules added one by one (16 cells at full load, 53k tokens/s offered, three seeds).

Configuration	AI tokens/s	Recoveries kept
Low-priority AI, no grants	48.4k	92.0%
+ grants, no AI next to a neural receiver	25.3k	97.4%
+ AI next to a neural receiver while two are free	41.3k	96.5%
+ AI next to a neural receiver while three are free (Backstop)	39.5k	97.4%
Backstop with a 70% share cap	34.9k	96.2%
Grants by deadline only, same priority, 70% share	22.8k	97.1%

them. With 20 cells at full load, five per GPU, Backstop allows only 128-token units next to the conventional receivers. It misses the layer-1 deadline for 0.03% of the TBs, as the run without AI, while the fixed 30% and 50% shares miss it for 0.10% and 0.23%. Backstop serves 27.2k tokens/s at 96.1% of the recoveries, and the 30% share with low-priority AI serves 17.0k at the same 96.1%.

GPUs. With four cells per GPU at full load, Backstop serves 8.8k, 19.4k, and 39.5k tokens/s on one, two, and four GPUs, and the recovered TBs grow from 53 to 110 and 202 per second. The free-receiver rule needs several neural receivers. With one GPU, a longer neural receiver run does not change which candidates find the neural receiver busy; every policy keeps all recoveries, and Backstop equals the 70% share with low-priority AI. With two and four GPUs, Backstop keeps 1.6 and 4.0 percentage points more recoveries than that share.

4.6 What each rule adds

Table 7 adds the rules one by one. Low priority alone serves the most AI and loses 8.0% of the recoveries. Granting AI only on GPUs without a running neural receiver keeps 97.4% and halves the AI served. The free-receiver rule returns most of the AI at the same recoveries: a neural receiver runs alone for 21% of the neural receiver time, and AI next to it costs no recovery. A share cap on top of low priority only removes AI. The last row grants by the deadline of the running neural receiver without low priority and without the free-receiver rule; it keeps the recoveries and serves 1.7× less AI than Backstop.

Where Recoveries Are Lost. We classify every recovery that a policy loses against the run without AI. For Backstop, 119 of 120 lost recoveries in two seeds are candidates that found no free neural receiver. The same holds for 272 of 278 with the 70% share with low-priority AI and for 313 of 337 with low priority alone. Late neural receiver runs dominate only without any protection: 1,452 of 1,849 with the fixed 100% share. The median neural receiver run takes 6.3 ms without AI, 7.0 ms under Backstop and under the 30% share with low-priority AI, 7.6 ms under the 70% share, and 9.1 ms under the fixed 100% share.

4.7 Choosing the TBs to recover

On the urban micro channel, sending every failed TB to the neural receiver takes 4,891-4,926 runs and recovers 663-757 TBs without AI. Sending only the TBs of slots with at most nine failed code blocks takes 651-777 runs and recovers 589-746 TBs: 85% fewer runs for 5% fewer recoveries. On the low-correlation channel with eight two-user cells, the neural receivers cannot take every failed TB. Sending only slots with at most three failed code blocks removes 40% of the runs and 22% of the recoveries and raises the recoveries per run from 0.77 to 0.99.

5 Related Work

GPU sharing in vRAN and AI-RAN. YinYangRAN [18] multiplexes LDPC decoding with other workloads by MPS shares that a learned predictor changes at intervals of at least one second. CloudRIC [19] pools accelerators for LDPC decoding, and Concordia [8] reserves capacity for vRAN tasks. AI-RAN studies [3, 12, 20, 21] place AI and RAN workloads across servers. Their radio work has one deadline and no second receiver. Backstop adds a recovery path with its own deadline and grants AI against both deadlines and against the free neural receivers.

GPU scheduling. DARIS, REEF, XSched, and nvtaskset [4, 5, 9, 22] schedule GPU stages, kernels, or contexts by priority, and Orion [23] runs best-effort GPU work when it does not interfere with a high-priority job. SMEC [25] sets MPS priorities on an edge server that is separate from the RAN server. These systems protect the latency of the running high-priority job. They have no notion of a TB that may still need a neural receiver or of a neural receiver that the next failed TB needs free. Backstop decides from the state of each TB and of every neural receiver, and our evaluation includes low priority and idle-only execution as baselines.

Neural receivers. DeepRx [10] and the NVlabs receivers [6, 24] design and train neural receivers, ARCHES [17] switches between an AI channel estimator and a conventional one, and OCUDU [15] exposes timing classes in the distributed unit. These works report receiver gains against their own baselines. We compare at equal LDPC effort with a production receiver and study the neural receiver as a workload that shares the GPU with AI.

6 Limitations

The channels are the synthetic channels of the public NVlabs configuration. The larger model allows four to five cells per GPU at full load; 32 and 48 cells run when half or two thirds of the cells are idle, and with 48 cells the layer-1 misses are 0.09% with and without AI. The recovery deadline follows from the TDD pattern and the PUSCH preparation time [2] and is not validated with a layer-2 stack; an 11.5 ms deadline delays the retransmission of an unrecovered TB by 5 ms. The baselines are the mechanisms of prior systems on our code: the share that follows the load has two or three shares and no load predictor, and it has no switching cost. The number of free neural receivers that the free-receiver rule requires is set by hand for the number of GPUs. A piece that is running when a condition stops holding runs to its end, and at low priority it can take twice its bound. Layer-1 misses are 0.03-0.04% of the TBs for Backstop and for the run without AI, which reflects the Python prototype and not a bound. Values are means over two or three seeds of 10-40 s runs.

7 Conclusion

A neural receiver that decodes more TBs than a production receiver at equal LDPC effort takes twice the uplink period. Backstop runs it only for the TBs the conventional receiver fails and that it is likely to decode, under a recovery deadline, and grants low-priority AI inference in pieces that keep the layer-1 deadline, the recovery deadline, and a free neural receiver for the next failed TB.

Under a changing load with mixed channels, Backstop serves 46.2k tokens/s while keeping 98.8% of the recoveries, $8.0\times$ a fixed GPU share and $1.3\times$ a share that follows the load with low-priority AI.

References

- [1] 3GPP. 2020. *Study on Channel Model for Frequencies from 0.5 to 100 GHz*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3173> Technical Report 38.901.
- [2] 3GPP. 2024. *NR; Physical Layer Procedures for Data*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3216> Technical Specification 38.214.
- [3] Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, and Yongguang Zhang. 2025. Distributed AI Platform for the 6G RAN. In *ACM Workshop on Open and AI RAN*. ACM, New York, NY, USA, 15–21. <https://doi.org/10.1145/3737900.3770167>
- [4] Amir Fakhim Babaei and Thidapat Chantem. 2025. DARIS: An Oversubscribed Spatio-Temporal Scheduler for Real-Time DNN Inference on GPUs. In *ACM/IEEE Design Automation Conference*. IEEE, Piscataway, NJ, USA, 1–7. <https://doi.org/10.1109/DAC63849.2025.11132423>
- [5] Joshua Bakita and James H. Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *ECRTS*, Vol. 335. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Wadern, Germany, 21:1–21:25. <https://doi.org/10.4230/LIPIcs.ECRTS.2025.21>
- [6] Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Andreas Oeldemann, Andreas Roessler, Timo Mayer, and Alexander Keller. 2023. A Neural Receiver for 5G NR Multi-User MIMO. In *IEEE Globecom Workshops*. IEEE, Piscataway, NJ, USA, 329–334.
- [7] Sebastian Cammerer, Reinhard Wiesmayr, Fayçal Ait Aoudia, Jakob Hoydis, Tommi Koivisto, Jakub Zakrzewski, Ruqing Xu, Pawel Morkisz, Chris Dick, and Alexander Keller. 2024. Real-time 5G NR Multi-user MIMO Receivers. <https://github.com/NVlabs/neural-rx>.
- [8] Xenofon Foukas and Bozidar Radunovic. 2021. Concordia: Teaching the 5G vRAN to Share Compute. In *ACM SIGCOMM*. ACM, New York, NY, USA, 580–596. <https://doi.org/10.1145/3452296.3472894>
- [9] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-Scale Preemption for Concurrent GPU-Accelerated DNN Inferences. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 539–558. <https://www.usenix.org/conference/osdi22/presentation/han>
- [10] Mikko Honkala, Dani Korpi, and Janne M. J. Huttunen. 2021. DeepRx: Fully Convolutional Deep Learning Receiver. *IEEE Transactions on Wireless Communications* 20, 6 (2021), 3925–3940. <https://doi.org/10.1109/TWC.2021.3054520>
- [11] Jakob Hoydis, Sebastian Cammerer, Fayçal Ait Aoudia, Avinash Vem, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. 2022. Sionna: An Open-Source Library for Next-Generation Physical Layer Research. arXiv:2203.11854 <https://arxiv.org/abs/2203.11854>
- [12] Haiyuan Li, Yulei Wu, and Dimitra Simeonidou. 2026. Deadline-Driven Hierarchical Agentic Resource Sharing for AI Services and RAN Functions in AI-RAN. arXiv:2605.07547 [cs.NI] <https://arxiv.org/abs/2605.07547>
- [13] NVIDIA Corporation. 2026. *Aerial CUDA-Accelerated RAN Documentation*. NVIDIA. <https://docs.nvidia.com/aerial/cuda-accelerated-ran/>
- [14] NVIDIA Corporation. 2026. *Multi-Process Service Documentation*. NVIDIA. <https://docs.nvidia.com/deploy/mps/>
- [15] Timothy O’Shea, Matthew Pennybacker, and Andriy Kharchenko. 2026. The OCUDU dApp Platform: An Open Runtime and E3 Interface for Real-Time AI-RAN. arXiv:2609.07843 [cs.NI] <https://arxiv.org/abs/2609.07843>
- [16] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115 <https://arxiv.org/abs/2412.15115>
- [17] Neagin Neasamoni Santhi, Davide Villa, Michele Polese, Salvatore D’Oro, Yunseong Lee, Koichiro Furueda, and Tommaso Melodia. 2026. ARCHES: Adaptive Real-Time Switching of AI Models for the RAN. arXiv:2604.23397 [cs.NI] <https://arxiv.org/abs/2604.23397>
- [18] Leonardo Lo Schiavo, Jose A. Ayala-Romero, Andres Garcia-Saavedra, Marco Fiore, and Xavier Costa-Perez. 2024. YinYangRAN: Resource Multiplexing in GPU-Accelerated Virtualized RANs. In *IEEE INFOCOM*. IEEE, Piscataway, NJ, USA, 721–730. <https://doi.org/10.1109/INFOCOM52122.2024.10621380>
- [19] Leonardo Lo Schiavo, Gines Garcia-Aviles, Andres Garcia-Saavedra, Marco Gramaglia, Marco Fiore, Albert Banchs, and Xavier Costa-Perez. 2024. CloudRIC: Open Radio Access Network (O-RAN) Virtualization with Shared Heterogeneous Computing. In *ACM MobiCom*. ACM, New York, NY, USA, 558–572. <https://doi.org/10.1145/3636534.3649381>
- [20] Syed Danial Ali Shah, Maryam Hafeez, Abdelaziz Salama, and Syed Ali Raza Zaidi. 2025. Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks. arXiv:2507.09124 [cs.NI] <https://arxiv.org/abs/2507.09124>
- [21] Syed Danial Ali Shah, Zeinab Nezami, Maryam Hafeez, and Syed Ali Raza Zaidi. 2025. The Interplay of AI-and-RAN: Dynamic Resource Allocation for Converged 6G Platform. In *IEEE INFOCOM Workshops*. IEEE, Piscataway, NJ, USA,

1–6. <https://doi.org/10.1109/INFOCOMWKSHPs65812.2025.11152939>

[22] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive Scheduling for Diverse XPU. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 671–692. <https://www.usenix.org/conference/osdi25/presentation/shen-weihang>

[23] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *ACM EuroSys*. ACM, Athens, Greece, 1075–1092. <https://doi.org/10.1145/3627703.3629578>

[24] Reinhard Wiesmayr, Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Jakub Zakrzewski, and Alexander Keller. 2024. Design of a Standard-Compliant Real-Time Neural Receiver for 5G NR. *arXiv:2409.02912* <https://arxiv.org/abs/2409.02912>

[25] Xiao Zhang and Daehyeok Kim. 2026. Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC. In *USENIX NSDI*. USENIX Association, Boston, MA, USA, 1759–1776. <https://www.usenix.org/conference/nsdi26/presentation/zhang-xiao>